CSA47 – Deep Learning **NAME: CH. Venkata chandu**
REG:192424140

CO1 – ASSESSMENT 1

CO1: *Mathematical foundations of deep learning – linear algebra, probability and information theory, numerical computation, machine learning basics, capacity, overfitting and underfitting, estimation, bias and variance, Bayesian statistics, stochastic gradient descent, challenges motivating deep learning.*

Assessment Title: Analyse the Mathematical and Machine Learning Foundations of a Crop-Disease Screening System (Total Marks - 50)

Nature of Assessment: Individual Duration: 1 Week Bloom's Level: K4 – Analyse
Submission Date: 13-10-2026

Scenario — "AgriVision" Crop-Disease Screening

AgriVision is an agri-tech start-up that helps farmers detect leaf diseases from smartphone photographs. Before the company invests in a deep neural network, its Chief Data Scientist wants a rigorous analysis of the mathematical and machine learning foundations of the problem.

You will work with a public leaf-disease image dataset of your choice (for example, the tomato subset of PlantVillage, with Healthy, Early Blight and Late Blight classes). Images may be resized to 64 × 64 grayscale, giving 4,096 raw pixel features per image.

The dataset may be used at increasing scales:

1,000 → 5,000 → 10,000 → full dataset images

You are required to examine how the data is represented, break down the sources of error and uncertainty in classical machine learning models, differentiate their optimisation and generalisation behaviour, and determine, with evidence, whether deep learning is justified. Every preprocessing step must be documented and examined for its effect on the results.

Design Challenge:

AgriVision now needs to support 38 crop–disease classes, add images whose labels are about 20% noisy, and run the classifier on a low-cost smartphone within 50 ms and 5 MB.

Analyse how these changes affect your findings and determine whether your conclusion should be retained, modified or redesigned.

Task 1 – Problem Structure and Data Representation

Learning task, input, output, performance measure

- **Task:** Supervised multi-class classification: learn f: x → y from labelled leaf photos.

- **Input:** A 64 × 64 grayscale image (4,096 pixel values). Colour is discarded, so colour cues such as yellowing are lost; texture and lesion shape remain. Smartphone photos vary in light and background, which constrains how robust the solution must be.
- **Output:** One of 3 classes (Healthy, Early Blight, Late Blight); predicted as a probability vector so uncertainty is visible to the farmer.
- **Performance measure:** Accuracy alone misleads under imbalance, so macro-F1 and recall per disease class are reported; missing a blight case is costlier than a false alarm. Training loss is cross-entropy, which differs from the evaluation metric.

Shapes of the data

Object	Shape	Type	Why it matters
One pixel intensity	1	Scalar	Basic unit, range 0–255
One image	64 × 64 (or 4,096)	Matrix / vector	Matrix keeps spatial neighbours; flattened vector loses them
Mini-batch of 32	32 × 4,096 (or 32 × 64 × 64 × 1)	Matrix / 4-D tensor	Enables one matrix product XW for the whole batch
Dataset	4,500 × 4,096	Matrix	Rows are samples, columns are features; covariance is 4,096 × 4,096
Labels	4,500 (integers) or 4,500 × 3 (one-hot)	Vector / matrix	One-hot is needed for cross-entropy and softmax gradient P − Y

Consequence: flattening makes 4,096 numbers that a linear model treats as unrelated, so a lesion shifted by a few pixels becomes a completely different vector. This is the main reason spatial structure matters later in Task 11.

L1, L2 and L∞ norms

Take x = (3, −4, 0, 1). L1 = 3+4+0+1 = 8; L2 = $\sqrt{(9+16+0+1)} = \sqrt{26} \approx 5.10$; L∞ = 4.

- L1 sums absolute values, treats all pixels equally and promotes sparsity; used for lasso and for robust distance (less sensitive to a few outlier pixels).
- L2 is smooth and differentiable; used for weight decay (ridge), k-NN Euclidean distance and PCA reconstruction error.
- L∞ looks at the single largest entry; used to detect saturated or extreme pixels and to bound the worst-case change in an input.

Effect of each preprocessing step

Step	Information gained	Information lost / risk
Resizing to 64 × 64	Fixed input size, 4,096 features, fast	Fine lesion detail and small spots are blurred away
Grayscale	One channel instead of three	Colour cues of disease
Normalisation (÷255, mean/std)	Similar feature scales, better conditioning, faster SGD	None if statistics come only from training data; leakage if computed on all data
Flattening	Works with classical models	Spatial neighbourhood structure
Train/val/test split (e.g. 70/15/15, stratified)	Honest estimate of generalisation	Fewer training samples; non-stratified splits distort class ratios

Task 2 – Feature Space using Linear Algebra

Hand-worked eigendecomposition

Consider a 2-pixel sub-problem with covariance C = [[2, 1],[1, 2]]. Characteristic equation: $\det(C - \lambda I) = (2-\lambda)^2 - 1 = 0$, so $\lambda_1 = 3$ and $\lambda_2 = 1$.

For $\lambda_1 = 3$: $(C - 3I)v = 0$ gives $v_1 = (1, 1)/\sqrt{2}$. For $\lambda_2 = 1$: $v_2 = (1, -1)/\sqrt{2}$. Check: trace = 4 = 3 + 1; det = 3 = 3 × 1.

Explained variance = $\lambda_i / \Sigma\lambda$: PC1 = 3/4 = 75%, PC2 = 1/4 = 25%. PC1 is the direction where the two neighbouring pixels rise and fall together (the correlated, redundant direction); keeping only PC1 loses 25% of the variance.

Spectrum and redundancy

Neighbouring pixels in a leaf photo are strongly correlated (smooth illumination, large leaf regions), so the eigenvalues of the 4,096 × 4,096 covariance decay very quickly: a handful of components hold most of the variance and the tail is almost zero. This shows that the 4,096 features are highly redundant and the data effectively lives in a much smaller subspace.

Eigenvalues, singular values and explained variance

For centred data matrix X (n × d) with SVD $X = U\Sigma V^T$, the covariance is $X^TX/(n-1) = V(\Sigma^2/(n-1))V^T$. Hence $\lambda_i = \sigma_i^2/(n-1)$, the right singular vectors are the principal directions, and explained variance ratio = $\sigma_i^2 / \Sigma\sigma_j^2$. Small singular values correspond to noise directions.

Reconstruction error and trade-off point

Reconstruction with k components has squared error equal to the sum of the discarded eigenvalues (Eckart–Young: this is optimal for rank k). The error drops steeply for small k and then flattens (an "elbow"). A sensible trade-off is the smallest k that retains about 95% of the variance, typically tens to a couple of hundred components rather than 4,096, giving a 20–50× compression with little visible loss.

Leading components as images

PC1 usually looks like a smooth illumination or overall brightness pattern of the leaf; the next few capture leaf shape/edges and the broad dark-light layout; later ones show finer texture, vein patterns and lesion-like blotches; the last ones look like noise. So the top components encode global brightness and shape, not disease, which hints that disease evidence sits in lower-variance directions (relevant to Task 10).

Computational cost

Forming the covariance costs $O(nd^2)$ and decomposing it $O(d^3)$ with d = 4,096 (about 7×10^{10} operations). Full SVD costs $O(n \cdot d \cdot \min(n,d))$. When n grows from 1,000 to the full dataset, the $O(nd^2)$ term grows linearly with n while d^3 stays fixed, so cost eventually becomes linear in n. Truncated/randomised SVD for k components costs about $O(n \cdot d \cdot k)$, which is much cheaper.

Task 3 – Uncertainty: Probability and Information Theory

Entropy of the class distribution

Class proportions: Healthy 1591/4500 = 0.354, Early Blight 0.222, Late Blight 0.424.

$H(p) = -\Sigma\, p \log_2 p = 0.354(1.50) + 0.222(2.17) + 0.424(1.24) \approx 0.530 + 0.482 + 0.524 = 1.537$ bits. Maximum for 3 classes is $\log_2 3 = 1.585$ bits.

Interpretation: the entropy is only slightly below the maximum, so imbalance is mild. A majority-class predictor would be correct 42% of the time, which is the baseline any model must beat. The lower the entropy (stronger imbalance), the more accuracy can be inflated by always predicting the common class.

Cross-entropy and KL divergence

$H(p, q) = H(p) + KL(p\|q)$. Cross-entropy is the average code length (or loss) when using the model q for data from p. A weak or underfitting model (e.g. linear on raw pixels) has cross-entropy well above H(p); an overfit model has low training cross-entropy but high validation cross-entropy, and is often overconfident (large loss on its few mistakes). The gap $KL = H(p,q) - H(p)$ is the extra uncertainty caused by the model, so comparing KL across models compares how far each is from the truth.

Asymmetry of KL (own numbers)

Let P = (0.8, 0.15, 0.05) (true) and Q = (0.4, 0.3, 0.3) (model).

$KL(P\|Q) = 0.8 \ln(2) + 0.15 \ln(0.5) + 0.05 \ln(1/6) = 0.554 - 0.104 - 0.090 = 0.361$ nats.

$KL(Q\|P) = 0.4 \ln(0.5) + 0.3 \ln(2) + 0.3 \ln(6) = -0.277 + 0.208 + 0.538 = 0.468$ nats.

The two differ (0.361 vs 0.468). KL(P‖Q) weights errors by the true probability: it punishes the model for giving low probability to things that really happen (mass-covering). KL(Q‖P) weights by the model: it punishes the model for placing mass where the truth has little (mode-seeking). Classifier training uses KL(P‖Q) because the true labels define P.

Cross-entropy minimisation = KL minimisation

Since $H(p, q) = H(p) + KL(p\|q)$ and H(p) does not depend on the model parameters, minimising cross-entropy over θ is the same as minimising $KL(p\|q_\theta)$. With one-hot labels this equals the negative log-likelihood, so it is also maximum likelihood, and therefore yields the same classifier.

Task 4 – Numerical Computation Issues

Naive softmax failure

$\text{softmax}(z)_i = e^{z_i}/\Sigma e^{z_j}$. For z = [1000, 999, 995], e^{1000} exceeds the float64 maximum ($\approx e^{709.78}$), giving inf, and inf/inf = NaN. For very negative z (e.g. −1000) every e^z underflows to 0, so the denominator is 0 and the result is 0/0 = NaN. In float32 the overflow threshold is only around $z \approx 88$.

Fix: subtract max(z) and log-sum-exp

Using $c = \max(z)$: $e^{z_i-c}/\Sigma e^{z_j-c} = e^{z_i}e^{-c}/(e^{-c}\Sigma e^{z_j})$, so the factor cancels and the result is unchanged. Shifted z = [0, −1, −5]; exponentials = [1, 0.3679, 0.0067]; sum = 1.3746; softmax = [0.7275, 0.2676, 0.0049], which sums to 1. There is no overflow since every exponent is ≤ 0, and no underflow of the denominator since the largest term is exactly 1.

Log-sum-exp: $\log \Sigma e^{z_j} = c + \log \Sigma e^{z_j-c}$. The log-softmax is $z_i - c - \log \Sigma e^{z_j-c}$, which avoids computing log(0) = −inf when a probability underflows. Cross-entropy should be computed from log-softmax, not log(softmax(z)).

Condition number and training

The condition number is $\kappa = \lambda_{max}/\lambda_{min}$ of the feature covariance. Because the PCA spectrum decays by several orders of magnitude, κ is huge for raw pixels (often 10^6 or more). The loss surface is then a long narrow valley: gradient descent needs a learning rate small enough for the steepest direction ($\eta < 2/\lambda_{max}$), yet progress along the flattest direction is proportional to $\eta \cdot \lambda_{min}$, so convergence takes about $O(\kappa)$ iterations, oscillates across the valley and is sensitive to learning rate. Standardising features and PCA whitening reduce κ and speed up training.

Task 5 – Baseline Learning Strategies

Three distinct strategies are used on raw (4,096-d) and PCA-compressed (e.g. 50–100-d) features:

Algorithm	Strategy	Hypothesis space / inductive bias	Raw 4,096-d	After PCA

Softmax regression	Linear parametric	Linear decision boundaries; classes assumed separable by hyperplanes in pixel space	4,096×3 weights, ill-conditioned, stable with L2; underfits nonlinear patterns	Faster, better conditioned, accuracy usually almost unchanged
k-NN	Non-parametric (instance-based)	Local constancy: nearby points share labels; no training	Distances become uninformative in 4,096-d; slow inference	Often improves (noise removed, distance more meaningful); much faster
Random forest / SVM-RBF	Tree-based / kernel	Axis-aligned piecewise-constant splits (RF); smooth kernel similarity (SVM)	RF copes with irrelevant pixels but splits on single pixels; SVM costs $O(n^2)$–$O(n^3)$	RF equal or slightly better; SVM much faster and often best

Why behaviour differs: linear models gain from PCA mainly through better conditioning and lower variance. k-NN depends entirely on the distance metric, so removing noisy low-variance directions helps it most. Trees pick one pixel at a time, so PCA components (which mix many pixels) give them more informative features, but they lose pixel-level interpretability. In all cases no method can recover spatial structure that was lost by flattening, which limits accuracy.

Task 6 – Capacity, Overfitting and Underfitting

Model	Capacity knob	Low capacity	High capacity
Polynomial / softmax	Degree, or regularisation strength λ (λ large = low capacity)	High train and validation error (underfit)	Train error → 0, validation error rises
k-NN	k (small k = high capacity)	Large k: over-smoothed, underfit	k = 1: memorises noise, overfits
Decision tree / RF	Max depth	Shallow tree: underfit	Deep tree: pure leaves, overfit

- **Pattern:** Training error decreases monotonically with capacity; validation error is U-shaped.
- **Causes:** Underfitting (left): the hypothesis space is too small to contain the true function, so both errors are high (high bias). Overfitting (right): the model fits noise and idiosyncrasies of the sample, so training error is tiny but validation error rises (high variance).
- **Gap:** The generalisation gap (validation − training error) is small at low capacity and grows with capacity; the best capacity is near the minimum of validation error, before the gap blows up. More data shifts the optimum to higher capacity and narrows the gap.
- **Test-set selection:** Choosing hyperparameters on the test set uses the test labels to make a decision, so the chosen model is the one that happened to do best on that particular sample. The reported error is optimistically biased and no longer an unbiased estimate of generalisation. Use a separate validation set, or k-fold cross-validation (e.g. 5-fold), which averages over several splits and uses data efficiently, and touch the test set once at the end.

Task 7 – Bias–Variance and Estimation

Bootstrap experiment

Draw $B \geq 10$ (e.g. 20) bootstrap resamples of the training set, train a low-capacity model (linear or k = 50) and a high-capacity model (k = 1 or deep tree) on each, and predict a fixed test set. For each test point: $\text{bias}^2 = (\text{mean prediction} - \text{truth})^2$, variance = variance of predictions across replicas; average over

test points. Expected result: the low-capacity model has high bias² and low variance; the high-capacity model has low bias² and high variance, consistent with the capacity curves in Task 6.

Does the sum match?

Expected error = bias² + variance + irreducible noise. The measured test error is usually close but not identical, because (1) only a finite number of bootstrap replicas is used, so the estimates are noisy, (2) bootstrap samples contain only about 63% unique points, which changes the variance slightly, (3) the noise term (label errors, ambiguous images) cannot be measured directly, and (4) for classification with 0-1 loss the decomposition is not exactly additive as it is for squared error.

MLE vs MAP of a class proportion

Estimate θ = proportion of Healthy leaves. Suppose a small sample has 7 Healthy of n = 10. MLE = k/n = 0.70. With prior Beta(2, 2) (mild belief that θ is near 0.5), MAP = $(k + \alpha - 1)/(n + \alpha + \beta - 2) = (7 + 1)/(10 + 2) = 0.667$.

Larger samples: k = 70, n = 100 gives MLE = 0.70 and MAP = 71/102 = 0.696. With n = 1,000, k = 700, MAP = 701/1002 = 0.6996.

The prior pulls the estimate toward 0.5 (shrinkage), which reduces variance at the cost of some bias; this is the same effect as L2 regularisation. As n grows the likelihood dominates and MAP → MLE: the prior acts like $\alpha + \beta - 2$ pseudo-observations, which are negligible against thousands of real ones.

Task 8 – Gradient-Based Optimisation (Softmax Regression)

Gradient derivation

Model: $z = W^T x + b$, $p_k = e^{z_k}/\Sigma_j e^{z_j}$. Loss for one sample with one-hot y: $L = -\Sigma_k y_k \log p_k$.

Derivative of softmax: $\partial p_k/\partial z_j = p_k(\delta_{kj} - p_j)$. Then $\partial L/\partial z_j = -\Sigma_k y_k (1/p_k)\, p_k(\delta_{kj} - p_j) = -\Sigma_k y_k(\delta_{kj} - p_j) = -y_j + p_j \Sigma_k y_k = p_j - y_j$ (since $\Sigma y_k = 1$).

By the chain rule, $\partial L/\partial W = x(p - y)^T$ and $\partial L/\partial b = p - y$. For N samples: $\nabla_W L = (1/N)\, X^T(P - Y)$ ($+ \lambda W$ with L2 regularisation), a 4,096 × 3 matrix.

Interpretation: the update is driven by the prediction error (P − Y), multiplied by the input features. Correct, confident predictions give $P - Y \approx 0$ and contribute nothing; wrong predictions give large corrections along their pixel pattern.

Batch vs stochastic vs mini-batch

Method	Gradient	Path	Noise	Cost per epoch
Batch GD	All N samples	Smooth, steady decrease	None	One update per epoch; each update costly
SGD (batch = 1)	1 sample	Very erratic, zigzag	High	N cheap updates; poor vectorisation
Mini-batch (e.g. 32)	32 samples	Slightly noisy but fast	Moderate	N/32 updates; best use of matrix operations, the standard choice

Noise in SGD slows exact convergence but gives faster early progress and helps escape plateaus; reducing the learning rate over time lets it settle.

Learning rates

- η too small (e.g. 0.0001): stable but very slow; loss barely decreases in the available epochs.

- η moderate (e.g. 0.01–0.1): fast and stable decrease, the best choice.
- η too large (e.g. 1 or more): loss oscillates or diverges, because η exceeds 2/λ_max along the steepest direction.

Link to conditioning (Task 4)

The Hessian of the softmax loss is proportional to the feature covariance, so a large κ forces a small stable η while flat directions barely move: training on raw unnormalised pixels is slow and oscillatory, whereas normalised or PCA-reduced inputs allow a larger η and converge in far fewer epochs.

Task 9 – Experimental Performance

Results are organised in one table per dataset size (1,000 → 5,000 → 10,000 → full). Template with expected patterns (fill the blank cells from your runs):

Model (feature)	Accuracy	Precision	Recall	F1	Train time	Inference time	Params	Memory
Softmax (raw)					low	very low	12,291	~0.1 MB
Softmax (PCA-100)					very low	very low	303	<0.01 MB
k-NN (raw)					none	high	stores all data	all data
k-NN (PCA-100)					none	medium	stores reduced data	small
RF / SVM (raw/PCA)					medium–high	medium	trees / support vectors	MBs

- **Patterns:** Accuracy rises with dataset size and flattens (diminishing returns); the gap between high-capacity and low-capacity models narrows as n grows.
- **Confusions:** Confusion is expected mainly between Early Blight and Late Blight (similar brown lesions and textures, harder to tell apart in grayscale at 64 × 64); Healthy is usually separated best because it lacks lesions.
- **Trade-offs:** PCA reduces training and inference time and memory substantially for all models, with a small accuracy loss for linear models and often a gain for k-NN. k-NN has the largest memory and inference cost since it stores the whole training set; the linear model has the smallest footprint.

Plots to include: accuracy vs dataset size per model; training time vs dataset size; confusion matrix of the best model; accuracy vs PCA dimension.

Task 10 – Theory vs Experiment

- **Explained variance vs retained accuracy:** Mostly yes for linear models: accuracy stays close to full-feature accuracy once ~90–95% variance is retained. Disagreement appears because variance is not the same as discriminative information: the highest-variance components capture lighting and leaf shape, whereas disease evidence can lie in smaller-variance directions. PCA is unsupervised and ignores the labels.
- **Optimisation speed vs condition number:** Mostly yes: raw features need a smaller learning rate and many more epochs than normalised/PCA features. Differences arise because the loss is not purely quadratic, mini-batch noise changes the effective path, and the softmax Hessian also depends on predicted probabilities, not just the covariance.

- **Bias–variance vs capacity curves:** Yes qualitatively: the high-bias model sits in the underfitting region and the high-variance model in the overfitting region. Quantitative differences come from the finite bootstrap, noise estimation and the non-additivity of 0-1 loss.
- **Causes of disagreement:** Finite samples, correlated pixels, non-quadratic losses, label noise, and near-duplicate images (PlantVillage has several photos of the same leaf, which can leak between train and test) make real behaviour deviate from clean theory.

Task 11 – Is Deep Learning Justified?

- **Remaining limitations:** Even after tuning, linear models are limited by their linear boundary, k-NN by high-dimensional distances, and RF/SVM by flattened pixels. Early vs Late Blight confusion remains and accuracy plateaus as data grows.
- **Curse of dimensionality, smoothness, manifolds:** In 4,096 dimensions a few thousand samples cover the space very sparsely, so k-NN neighbours are not truly near and distances concentrate. Local constancy fails because a shifted or rotated leaf is far away in pixel space yet has the same class. Manifold structure: the PCA spectrum shows the data lies near a low-dimensional manifold, but leaf deformations and illumination curve it, so a linear projection cannot flatten it.
- **Evidence for hierarchical representation:** Disease cues are local, compositional patterns (edges → spots → lesion regions) that can appear at any position. Classical models on flattened pixels cannot share parameters across positions or build such hierarchies. The slow gain with more data and the persistent confusion of similar lesions suggest that better features, not bigger classical models, are needed.

Aspect	Classical ML	Deep network (CNN)
Data need	Small–medium	Large (augmentation / transfer learning helps)
Compute	Cheap, CPU	GPU for training
Interpretability	Higher (weights, PCA images)	Lower (needs saliency maps)
Feature engineering	Manual / PCA	Learned automatically

Conclusion: deep learning is justified because the evidence above (plateau of classical models, Early/Late Blight confusion, loss of spatial structure through flattening, curved low-dimensional manifold) points to the need for learned, translation-aware hierarchical features. A small CNN is recommended, with the classical model kept as a baseline.

Task 12 – Design Challenge

Impact Analysis

- **What changed:** Classes 3 → 38; about 20% of labels are wrong; deployment limits of 50 ms and 5 MB are new.
- **Larger label space:** Maximum entropy rises from $\log_2 3 = 1.58$ to $\log_2 38 = 5.25$ bits, and chance-level cross-entropy from $\ln 3 = 1.10$ to $\ln 38 = 3.64$ nats. Random-guess accuracy falls from 33% to 2.6%, linear models need 38 weight vectors, and fine-grained classes (several diseases on one plant) are harder to separate.
- **Label noise:** Noisy labels act like extra irreducible noise: observed error is at least about the noise rate (~20%) even for a perfect model, so the estimated validation error is biased upward. High-capacity models memorise the wrong labels, so variance increases; low-capacity models are more tolerant.

- **Broken assumptions:** Labels are no longer reliable, classes are no longer few and separable, memory and compute are no longer unconstrained (k-NN storing all data, SVM with many support vectors), and the original data size may not suffice per class.

Examination of the Original Conclusion

- **Fit within limits:** A linear softmax on PCA features (e.g. $100 \times 38 + 38 \approx 3{,}838$ parameters ≈ 15 KB) easily fits 5 MB and runs in well under 1 ms. k-NN storing $4{,}500 \times 4{,}096$ bytes ≈ 18 MB breaks the memory limit, and large RF/SVM models usually break either limit. A CNN of about 0.5–1 M parameters (2–4 MB in float32, about 1 MB in int8) fits both. Confirm with your own measured timings.
- **Sensitivity:** Most sensitive: k-NN and deep trees (memorise noisy labels, unreliable distances) and high-capacity models in general. Accuracy for visually similar classes drops most.
- **Less affected alternative:** Regularised low-capacity models and ensembles (RF averaging) are less affected by noisy labels, because errors are averaged out rather than memorised; a small CNN with a noise-robust loss gives the best combination of accuracy and size.

Adaptation Options

Option	Capacity	Error	Cost
Smaller PCA dimension / stronger L2	Decreases	Lower variance and less noise memorisation; bias increases if too strong	Fewer parameters; faster and smaller
Noise-robust loss or label smoothing ($\varepsilon \approx 0.1$–0.2)	Effectively lowers confident fitting	Reduces overconfidence and influence of wrong labels; improves calibration	Almost none
Compress / simplify model (small CNN, pruning, int8 quantisation)	Reduced but structured	Small accuracy loss, may improve robustness	Meets 5 MB / 50 ms; extra tuning work

Evidence-Based Conclusion

Decision: MODIFY towards a redesign. Keep the analysis pipeline (normalisation, PCA, cross-entropy training, validation-based selection) but replace the classical best model with a compact CNN.

- **Mathematics:** Entropy rises to 5.25 bits and chance accuracy falls to 2.6%, so a linear model on flattened pixels is too weak; the manifold and local-constancy limits seen in Task 11 get worse with 38 classes.
- **Bias–variance:** With 20% label noise, high-variance models (k-NN, deep trees) degrade most (Task 7), while regularised models stay stable.
- **Conditioning:** The Hessian conditioning (Task 4) still favours normalised inputs and a tuned learning rate for training the new model.
- **Resource measurements:** Only the linear model and a compact CNN satisfy 5 MB and 50 ms; k-NN does not.
- **Trade-off:** Use label smoothing or a robust loss, early stopping, augmentation and int8 quantisation to trade a little accuracy for robustness and deployability. Final choice: a quantised small CNN with label smoothing, with the PCA + softmax model as fallback baseline.